

Lab 6 Report: Path Planning

Team 4

Christian Cassamajor-Paul (ed.)

Alicia Ramirez

Leo Sun

Raymond Wang

RSS

April 25, 2026

1 Introduction (Christian)

Among the most important capabilities for autonomous systems are path planning, creating a trajectory through a given environment; path following, executing controls to follow a given trajectory; and localization, knowing the vehicle's location and heading. The combination of these capabilities within a single platform enables real-time autonomous vehicle navigation. In this lab, we first investigate the performance of two different path planning approaches towards that end: the sample-based approach, RRT-Connect, and the ubiquitous grid-based approach, A*. Additionally, we then implement RRT-Connect along with a Pure Pursuit controller and Monte Carlo Localization (MCL) system to create a full autonomous driving stack. Using a map given beforehand, our Monte Carlo Localization estimates the car's 2D pose using noisy odometry and LiDAR measurements. This state-estimation layer acts as the bridge between sensing and planning/control. The RRT-Connect planning layer then takes in the pose estimate from the MCL, along with a desired goal point, to quickly generate a feasible trajectory for the Pure Pursuit controller to follow. The purpose of this lab was to explore path planning approaches and subsequently implement, tune, and evaluate a full autonomous navigation and driving pipeline (see Figure 1) in both simulation and on the physical car. Each step has unique challenges. For planning, the main hurdles include ensuring computational efficiency and creating a trajectory that abides by the physical constraints of hardware and the environment, i.e., a feasible trajectory. For the controller, the main challenge involved tuning the look-ahead parameter to an appropriate value. For localization, challenges include overcoming odometry drift, processing noisy LiDAR scans, and localizing in spaces with few distinct features or with features

that look alike. Our final design for the racecar implementation involves RRT-Connect for extremely quick trajectory generation, albeit at the expense of optimality, an empirically tuned Pure Pursuit controller with fixed look-ahead, and particle filter localization that advances multiple pose hypotheses through a motion model, reweights, and then resamples them according to a sensor model for convergence to the most likely pose estimate.

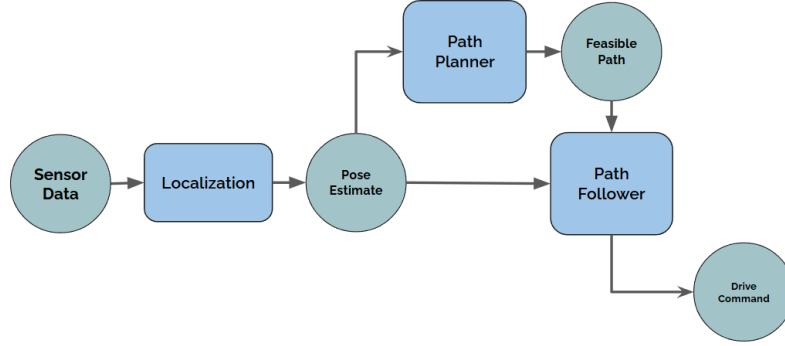


Figure 1: Diagram of the autonomous navigation pipeline implemented. Blue squares represent capabilities that use and/or produce the various inputs and outputs (green circles).

2 Technical Approach

2.1 Localization (Christian and Leo)

As covered in Lab 5, our Monte Carlo Localization system estimates the car’s 2D pose (x, y, θ) —where (x, y) is position and θ is yaw—from noisy odometry and LiDAR measurements against a known occupancy grid map. The filter maintains N particles, each a pose hypothesis, which are initialized by sampling Gaussian noise with a standard deviation of 0.1 around a user-provided pose from the RViz 2D Pose Estimate button. These hypotheses are then routinely updated by passing the particles through a motion model and sensor model, and the mean hypothesis of the resulting distribution is taken to be the best estimate of the car’s pose.

2.1.1 Motion Model

On each odometry callback, we compute a body-frame control increment $\mathbf{u}_t = [\Delta x, \Delta y, \Delta \theta]^\top$, where Δx and Δy are the body-frame linear displacements and $\Delta \theta$ is the yaw change over the elapsed time step. This increment is applied to

each particle in its own frame:

$$x_t = x_{t-1} + \Delta x \cos \theta_{t-1} - \Delta y \sin \theta_{t-1}, \quad (1)$$

$$y_t = y_{t-1} + \Delta x \sin \theta_{t-1} + \Delta y \cos \theta_{t-1}, \quad (2)$$

$$\theta_t = \theta_{t-1} + \Delta \theta, \quad (3)$$

followed by additive Gaussian noise per component to capture odometry uncertainty.

2.1.2 Sensor Model

For each particle, we ray-cast the LiDAR beams through the map and generate the ranges that would be expected if the particle’s pose were correct. These expected ranges are then compared to the actual LiDAR measurement to score how well the particle explains the observation. The likelihood of a single measured range given its corresponding expected range is modeled as a mixture of four components: a Gaussian centered at the expected range (correct hits), a decaying term for shorter-than-expected returns (unmapped obstacles), a spike at the maximum range (sensor failures), and a uniform term (random returns):

$$p(z \mid d) = \alpha_{\text{hit}} p_{\text{hit}} + \alpha_{\text{short}} p_{\text{short}} + \alpha_{\text{max}} p_{\text{max}} + \alpha_{\text{rand}} p_{\text{rand}}, \quad (4)$$

with empirically tuned weights (0.69, 0.12, 0.07, 0.12) respectively. The full mixture is precomputed into a lookup table for fast indexing at runtime. To score a particle, we combine the per-beam likelihoods across $B = 100$ downsampled LiDAR beams using a geometric mean, which prevents a few outlier beams from collapsing the weights onto a handful of particles:

$$w_i = \exp \left(\frac{1}{B} \sum_{b=1}^B \log p(z^b \mid d_i^b) \right). \quad (5)$$

2.1.3 Update Loop

Particle weights are normalized across the cloud, and resampling is triggered on roughly half of sensor updates to balance convergence against particle impoverishment; between resamples, likelihoods are accumulated multiplicatively. The final pose estimate is the weighted mean of particle positions, with `atan2` applied to the weighted sums of $(\sin \theta, \cos \theta)$ to handle angle wrapping.

2.2 Path Planning (Christian and Leo)

We explored two different path planning algorithms, one probabilistic and the other deterministic. However, there are some design choices and helper functions implemented universally across both planners. Before running either planner, we first inflate the Stata basement occupancy grid by a radius of around 0.45m to account for the racecar’s footprint and add a safety buffer. Additionally, we convert from (u,v) cell coordinates to (x,y) world coordinates and back according

to the following equations and using the map origin (x_o, y_o, θ_o) and resolution r given in the occupancy grid metadata:

$$u(x, y) = \lfloor \frac{1}{r} \rfloor (\cos \theta_o (x - x_o) + \sin \theta_o (y - y_o)), \quad (6)$$

$$v(x, y) = \lfloor \frac{1}{r} \rfloor (-\sin \theta_o (x - x_o) + \cos \theta_o (y - y_o)), \quad (7)$$

$$x(u, v) = r(\cos \theta_o u - \sin \theta_o v) + x_o, \quad (8)$$

$$y(u, v) = r(\sin \theta_o u + \cos \theta_o v) + y_o, \quad (9)$$

$$(10)$$

2.2.1 Sampling Based Approach: RRT-Connect (Leo)



Figure 2: RRT-Connect Tree Development. Left picture showcases intermediate step as both trees (in red) stretch towards each other. Right picture shows finished connected tree and the resulting smooth path (white line).

We implemented the bidirectional RRT-Connect algorithm, which simultaneously grows two rapidly-exploring random trees—one rooted at the x, y start location q_{start} and one rooted at the x, y goal location q_{goal} —until they meet and join together. Compared to a unidirectional RRT, this formulation typically reduces planning time because the effective exploration frontier doubles and the two trees greedily bias toward one another on every iteration.

At each iteration, the planner samples a random point $q_{\text{rand}} \in R^2$ by uniformly selecting a free cell (u, v) from the inflated occupancy grid and converting its center to world coordinates. Sampling only from free cells avoids wasting iterations on trivially infeasible configurations.

The active tree T is then extended toward q_{rand} . We first locate the nearest existing node q_{near} in T by minimizing squared Euclidean distance:

$$q_{\text{near}} = \arg \min_{q \in T} \|q - q_{\text{rand}}\|^2 \quad (11)$$

Next, we generate a candidate node q_{new} by stepping from q_{near} toward q_{rand} by at most a fixed step size $\Delta = 0.5$ m:

$$q_{\text{new}} = \begin{cases} q_{\text{rand}} & \text{if } \|q_{\text{rand}} - q_{\text{near}}\| \leq \Delta \\ q_{\text{near}} + \Delta \cdot \frac{q_{\text{rand}} - q_{\text{near}}}{\|q_{\text{rand}} - q_{\text{near}}\|} & \text{otherwise} \end{cases} \quad (12)$$

Before adding q_{new} , the segment $(q_{\text{near}}, q_{\text{new}})$ is collision-checked by sampling intermediate points at a resolution of $\delta_c = 0.1$ m and verifying that each lies in free space on the inflated map. If any sample is occupied, the extension is discarded; otherwise, q_{new} is appended to T with q_{near} recorded as its parent.

Whenever this extension succeeds, the opposing tree attempts to connect to q_{new} by repeatedly applying the same nearest-node, steering, and collision-check procedure in a tight loop. Unlike standard RRT, which extends by only one step per iteration, RRT-Connect greedily continues extending the second tree until it either reaches q_{new} or is blocked by an obstacle. This aggressive connect step is the main source of the algorithm’s practical speedup.

When the opposing tree successfully reaches q_{new} , the raw path is extracted by tracing parent pointers from the connection node back to each tree’s root, reversing one branch, and concatenating the two resulting sequences.

To improve the typically jagged output of sampling-based planning, we apply 200 iterations of random shortcutting. In each iteration, two waypoints (p_i, p_j) with $i < j - 1$ are selected uniformly at random and, if the straight-line segment between them is collision-free, the intermediate subpath is replaced by that shortcut. The final smoothed path is published as a **PoseArray** for the Pure Pursuit controller.

It is important to note that while RRT-Connect is guaranteed to find a path as time continues to infinity (i.e, it is probabilistically complete), for practical purposes, we place an upper bound of $t = 10000$ iterations on the algorithm.

2.2.2 Grid Based Approach: A* (Christian)

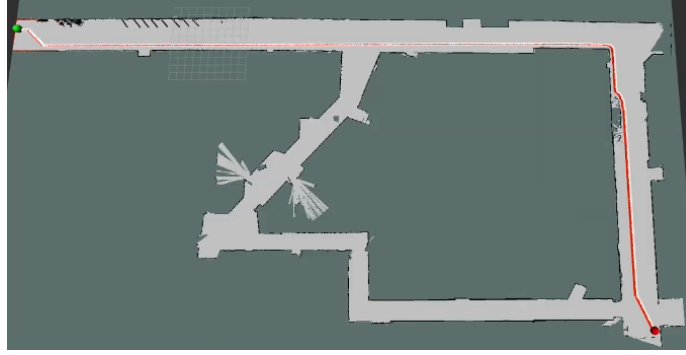


Figure 3: Picture of completed A* search with resulting path in red.

In addition to the sample-based approach, we explored an implementation of A* search, one of the most popular grid-based search algorithms used today. The A* algorithm is an evolution of Dijkstra’s algorithm which uses a map of the environment and a cost function $f(q)$ to determine the shortest path to a specified goal. On every step of the search, the algorithm chooses the next step in the path based on whichever candidate has the smallest cost function value. The cost function $f(q) = g(q) + h(q)$, where $g(q)$ measures the distance along the current potential trajectory from the start to q , and the heuristic $h(q)$ estimates the distance from q to the goal. With the correct choice of heuristic, the algorithm is guaranteed to be resolution-complete and resolution-optimal.

In our implementation, both $g(q)$ and $h(q)$ utilize euclidean distance $\|q_1 - q_2\|$. By measuring straight-line distance, our heuristic will never overestimate the distance, and as such, it is an admissible heuristic. This means that our A* implementation is optimal at the resolution provided by the occupancy grid.

Once given the inflated map, start pose q_{start} , and end pose q_{end} , the A* search algorithm begins by initializing priority queue with q_{start} with a priority/cost of 0, dictionary $D_{cost_so_far}$ which progressively maps explored q ’s to their $g(q)$ values, as well as dictionary D_{came_from} that tracks connections between cells by storing a given path point q_n as a key and its predecessor q_{n-1} as the corresponding value. The use of a priority queue massively speeds up the search process as the smallest cost, and therefore shortest paths are explored first.

On each step of the algorithm, $q_{current}$ is popped from the priority queue and a goal check is performed, i.e, does $q_{current} = q_{end}$? If yes, the path is extracted from D_{came_from} and published as a **PoseArray** for the controller. If $q_{current}$ is not the desired goal position, we get a list of neighbor cells to explore. This list comes from taking the 8 neighbor cells in a 3x3 grid around $q_{current}$ that are

not occupied, unknown, or out of bounds. For each valid neighbor $q_{neighbor}$, we calculate the cost $g(q_{neighbor})$ going through $q_{current}$:

$$g(q_{neighbor}) = D_{cost_so_far}[q_{neighbor}] + \|q_{neighbor} - q_{current}\| \quad (13)$$

If the neighbor has not yet been explored, or if its new cost is less than before, then we add it to the priority queue with a priority value equal to $f(q_{neighbor})$ and update both dictionaries appropriately, ending the current iteration.

This process continues until all possible options have been exhausted or the shortest path to the goal is found. If no path exists, the algorithm will return an empty list and print a message that no path could be found. As a result, in addition to being resolution-optimal, this algorithm is also resolution complete.

2.3 Path Following - Pure Pursuit Controller (Alicia)

Once a trajectory is provided by the path planner, the path follower processes the input path, calculates a goal position to track along the trajectory based on the look-ahead distance L , calculates steering angle γ with the pure pursuit controller, and finally publishes that angle to the Ackermann.

On each odometry update, we receive the current pose of the robot and check whether the robot is 0.5 meters away (Euclidean distance) from the end of the path. If it is not, we iterate over each line segment in the trajectory to find the line segment that is closest to the pose of the robot. As seen in the figure below, a vector between two points on the trajectory is calculated (U), and the vector between the first point and the robot is calculated (V). Then we perform a dot product to project the robot onto the line segment, which becomes the closest point on the line. That point is then compared against the pose to calculate the distance.

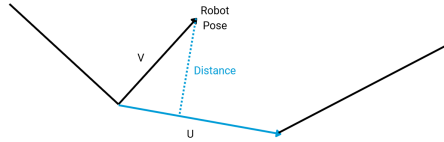


Figure 4: Calculation of distance of robot pose to closest line segment

We then create a circle centered of radius $L = 0.5$ meters centered around the current pose and check to see if and where this circle intersects with the closest line segment. If there is no intersection, the process is repeated with the next closest line segment. If they do intersect, the intersection is converted into the robot's frame, and that becomes the next goal point, aka "current goal", for the pure pursuit.

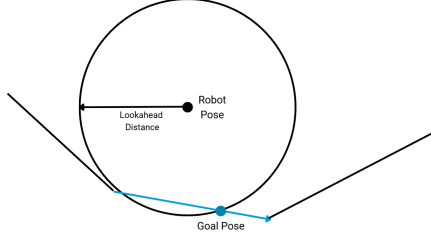


Figure 5: Intersection of look ahead radius with the closest line segment on the trajectory to find the goal pose

The goal point calculated above is then fed into the pure pursuit controller, a pre-defined approach to driving, which calculates the steering angle γ using the formulas below:

$$\alpha = \tan^{-1}\left(\frac{\text{goal point } y}{\text{goal point } x}\right) \quad (14)$$

$$\gamma = \tan^{-1}\left(\frac{2 * \text{wheelbase length} * \sin(\alpha)}{\text{look-ahead distance}}\right) \quad (15)$$

The look-ahead distance is essential to a well-performing controller. Too big, and the robot will cut corners. Too small, and the robot cannot adjust for turns quickly enough. We determined our value of $L = 0.5$ meters via extensive testing throughout Lab 3. As the physical hardware and velocities we are employing are extremely similar, we decided to use the same value.

3 Experimental Evaluation (Raymond)

Given previous (Lab 3) in-depth isolated controller evaluation, we are omitting that discussion in this report. Similarly, for brevity, we also omit evaluation of simulation localization (see Lab 5) as the real-life results covered are more relevant. Instead, this section focuses on path planning, physical localization, and the full navigation pipeline in simulation and real life.

3.1 Path Planner Comparison: A* vs. RRT-Connect

To evaluate our two path planning implementations, we designed experiments comparing A* and RRT-Connect across three metrics: path length (m), planning time (s), and minimum distance to obstacle (m). For each trial, both planners were given identical start and goal poses on the same inflated occupancy grid map of Stata basement. Five trials were conducted with varied start/goal configurations spanning short, medium, and long routes. Path length was computed by summing Euclidean distances between consecutive waypoints in the returned trajectory. Planning time was measured as wall-clock time from goal submission to trajectory publication. Minimum obstacle distance was

computed by querying the inflated map at each waypoint and recording the minimum clearance observed.

Results are summarized in Table 1 and visualized in Figure 6.

Trial	A* Length (m)	RRT Length (m)	A* Time (s)	RRT Time (s)	A* Min Dist (m)	RRT Min Dist (m)
1	61.29	104.26	2.14	0.33	0.4536	0.7531
2	33.15	31.94	0.91	0.01	0.4536	0.6766
3	57.71	59.38	1.96	0.24	0.4536	0.4594
4	18.97	77.97	0.08	0.00	0.4536	1.5793
5	48.29	102.22	1.74	1.46	0.4536	0.5090

Table 1: Comparison of A* and RRT path length, planning time, and minimum obstacle distance across five trials.

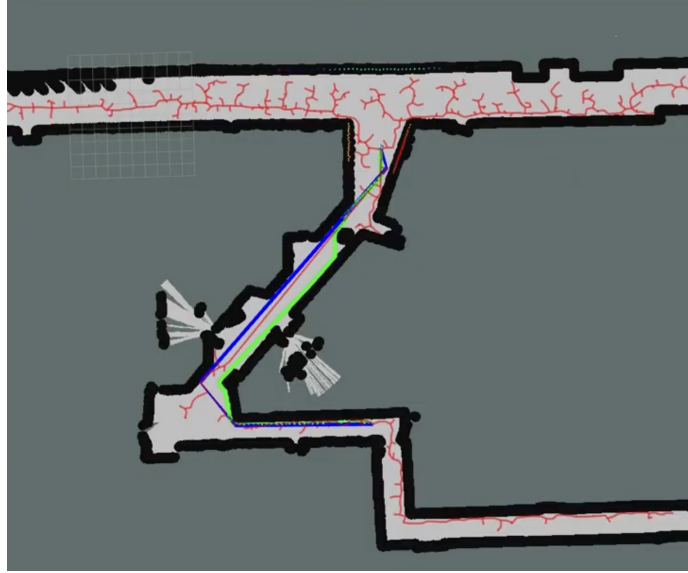


Figure 6: Overlay of RRT-Connect (red tree, blue optimized path) and A* (green) trajectories on the Stata basement occupancy grid. The A* path follows tighter, more direct routes through corridors, while the RRT tree explores broadly before converging.

On average, A* produced paths that were 33.7% shorter than RRT-Connect, confirming its optimality guarantee on the discretized grid. This is most pronounced in Trials 1 and 4, where RRT-Connect paths were 70% and 310% longer respectively — likely due to the planner sampling into suboptimal regions before the two trees connected. RRT-Connect, however, planned 77.5% faster on average, with near-zero planning times in Trials 2 and 4, demonstrating its advantage in environments where planning speed is prioritized over path quality.

Regarding safety, A* maintained a minimum obstacle clearance of 0.4536 m across all trials, a direct consequence of the fixed map inflation. RRT-Connect clearances were more variable (0.46–1.58 m), with Trial 3 approaching the A* minimum. This suggests that while RRT-Connect can incidentally produce safe paths, the safety guarantee is weaker without explicit inflation or post-processing. We decided to ultimately continue tests using RRT path planning. We suspected that our final challenge may in part require multiple instances of path planning in reasonably quick succession, and as a result, prioritized computation time. Furthermore, the RRT length increase is primarily a result of the fact that random sampling tends to miss the narrow but efficient passageway between two pillars near the basement classroom. We did not suspect encountering similar situations on Johnson track, or other environments, and so discounted this heavily, as did we discount the distance to obstacle clearance given our safety controller and inflated map.

3.2 Localization Validation

A critical dependency of the Pure Pursuit controller is the quality of the pose estimate supplied by the upstream localization module. Because the pose callback directly determines the look-ahead point computation, any error or latency in localization propagates immediately into tracking error. In our prior localization experiments (Lab 5), we found that our particle filter produced reliable pose estimates along straight corridors and turns. As a reminder, to evaluate localization accuracy on the physical car, we designed two repeatable tests: a straight-line drive and a closed-loop circular drive. In both cases, we compared the particle filter’s pose estimate against ground-truth positions measured with a tape measure and translated into map coordinates using the Foxglove measurement tool. Each test was repeated across five trials. In the straight-line test, the car was driven forward from a known start to a marked endpoint, and we computed the Euclidean error between the estimated and measured positions. In the circular test, the car was driven around the orange circle in the Stata basement and returned to its starting position; any discrepancy between the start and end estimates reflects accumulated drift over the loop.

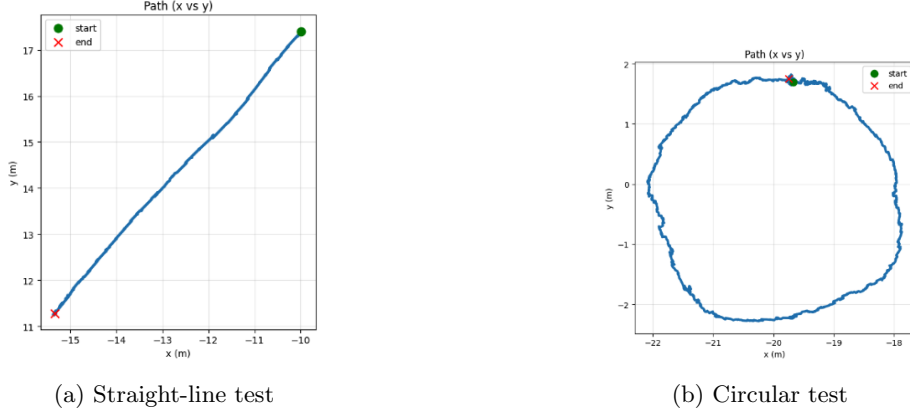


Figure 7: Real-world localization test examples.

Across both tests, the particle filter achieved an RMSE of approximately 9 cm, and qualitatively was able to accurately track the position of the racecar in a loop around Stata basement. Furthermore, as the car was driven manually via teleop in the above tests, it did not stop exactly on the marked endpoint in every trial, so the true localization accuracy is likely slightly better than reported.

3.3 Integrated System Performance

Having validated each component independently and using known valid parameters for localization and controller, we focused on evaluating the full pipeline from path planner to controller first in simulation and then on the physical racecar. The key metric throughout is cross-track error (CTE), defined as the perpendicular distance from the robot’s current pose to the nearest point on the planned trajectory:

$$E = |\delta y_{\text{robot}}^{\text{line}}| \quad (16)$$

CTE was logged at every pose callback invocation and recorded as a time series, allowing us to correlate tracking error with specific path geometries (e.g., corners vs. straights).

3.3.1 Simulation

In simulation, the mean CTE across a full traversal of the Stata basement was **0.0188 m** (Figure 8), with the majority of time steps remaining below 0.05 m. Transient spikes up to 0.175 m were observed at sharp turns, where the look-ahead circle intersects a geometrically discontinuous segment, and the commanded steering angle jumps discontinuously. Because the simulator provides noiseless, ground-truth pose, these spikes reflect purely the geometric limitations of the Pure Pursuit algorithm rather than any sensing or localization error, establishing a best-case performance baseline.

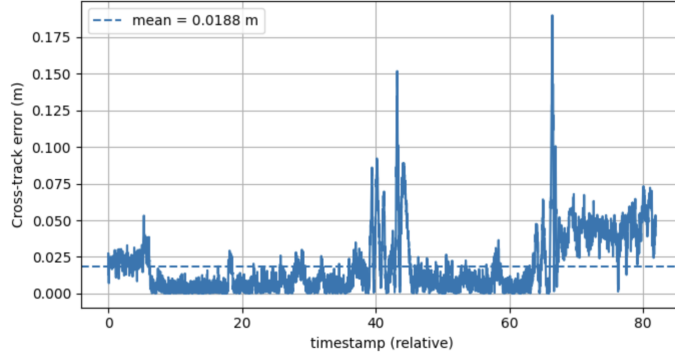


Figure 8: Cross-track error over time for simulation trial around Stata basement. The dashed line indicates the mean CTE of 0.0188 m. Periodic spikes correspond to sharp geometric turns in the A*-planned path and new goal points. Performance across other 5 tests were not materially different.

3.3.2 Physical Racecar

We then deployed the same planner/controller stack on the physical racecar, using particle filter localization to supply pose estimates to the Pure Pursuit pose callback.

To characterize real-world performance more rigorously, we segmented CTE measurements by path geometry — corners and long straights — and computed mean error for each. Results across representative segments are shown in Figure 9.

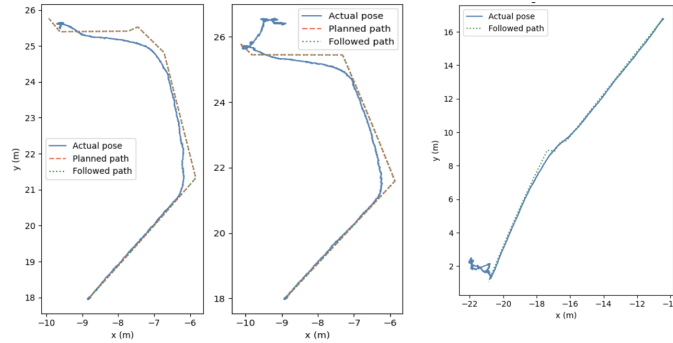


Figure 9: Real-world trajectory tracking across three path segments. Each plot shows the planned path (dashed), followed path (dotted), and actual pose (solid). Left and center: two corner segments. Right: a long straight.

On long straight segments, the mean CTE across all trials was approximately **0.03 m**, only marginally above the simulation baseline of 0.0188 m. This result indicates that on straight paths, the additional noise introduced by real-world

actuation and localization is modest, and the Pure Pursuit controller tracks effectively. On corner segments, mean CTE rose to approximately **0.05 m**. This degradation is expected: corners require rapid changes in steering angle, and any lag in the pose estimate — whether from particle filter latency or communication delay — causes the look-ahead point to be computed from a slightly stale pose, producing over-correction or undercutting of the turn.

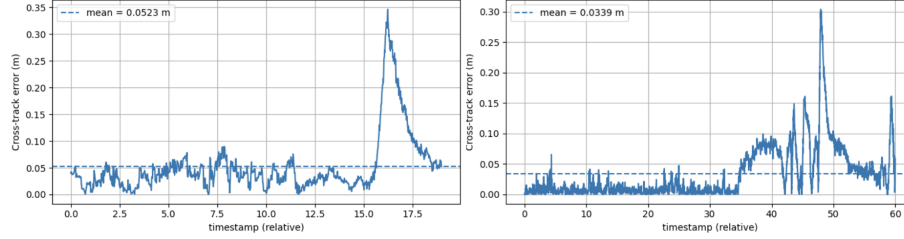


Figure 10: CTE in the corner and straight real world tests respectively. Results from other trials did not differ materially.

4 Conclusion (Leo)

In this lab, we built a complete autonomous navigation stack for our racecar by integrating localization, path planning, and trajectory tracking into a single pipeline. On the planning side, we implemented and compared two algorithmically distinct approaches: a grid-based A* search that guarantees resolution-optimal paths on a discretized occupancy grid, and a sampling-based RRT-Connect planner that grows two trees bidirectionally and post-processes the result with random shortcutting. Our experiments showed trade-offs between these two methods: A* produced paths that were on average 33.7% shorter and maintained uniform clearance from inflated obstacles, while RRT-Connect planned roughly 77.5% faster at the cost of more variable path quality. We ultimately moved forward with RRT-Connect because we anticipated that our downstream use case would benefit from faster re-planning, and our safety controller and inflated map mitigate the weaker clearance guarantees of sampled paths. On the control side, we implemented a geometric Pure Pursuit controller and validated it end-to-end, measuring a mean cross-track error of 0.0188 m in simulation, approximately 0.03 m on real straights, and 0.05 m on real corners. The discrepancy between straights and corners corresponds to regions where pose-estimate latency and rapid steering changes compound, highlighting the tight coupling between the three components of the stack.

The skills developed here map directly onto problems faced by real autonomous systems. Planning on inflated occupancy grids, balancing optimality against planning latency, and tracking trajectories under noisy pose estimates are core challenges in self-driving cars, warehouse robots, and mobile service robots, all of which must plan and re-plan continuously in partially known environments.

Several directions would strengthen the system. On the planner side, an asymptotically optimal variant such as RRT*, or a heuristic-guided hybrid that seeds A* with sampled waypoints, would combine speed with optimality. On the control side, adaptive look-ahead based on local curvature and velocity, or a switch to a model-predictive controller, would reduce corner error. Finally, fusing IMU data into the particle filter would bridge scan-match failures during sharp turns and directly reduce the cornering CTE that currently dominates real-world tracking error.

5 Lessons Learned

Christian Cassamajor-Paul

I learned that putting the extra bit of effort into prep work and planning before implementation is extremely valuable. This work provides high-level design choices and direction that can save many headaches as you progress through building your project. This is especially useful when working on large-scale projects with many moving parts and interfaces.

Alicia Ramirez

I learned that good evaluation of previous parts allows you to rely on them when building up and making more complex code.

Leo Sun

I learned that even if you develop individually working components of a system, it doesn't guarantee that the final pipeline will work. When combining our planner, controller, and localization, there was a lot of debugging to be done.

Raymond Wang

I learned that active collaboration despite the parallelization of work makes integration much quicker. When you fully appreciate the nuance of your teammate's implementations, you are better prepared to efficiently combine them into a comprehensive solution that leverages all their strengths.